

APSP- All Pairs Shortest Paths

We can solve this problem using **Belman-Ford** by executing it from *each vertex*

The cost of this solution is:
 $\Theta(|V|^2|E|) = O(|V|^4)$

The **output** required for this problem is a 2-dimensional matrix of size $\Theta(|V|^2)$

	1	2	...	j	...	V
1						
2						
⋮				⋮		
i			...	$\delta(i, j)$	...	
⋮				⋮		
V						

Dynamic Programming



We use new strategy to
do much better...

Development of Dynamic Programming Algorithm

1. Optimal solution structure specification
2. Recursive formalization of optimal value
3. Bottom-up computation of optimal value (using a table)
4. Optimal solution construction using the computed data

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Cost

Command Number	Cost
1	$\Theta(V)$
2	$\Theta(V ^2)$
3-6	$\Theta(V ^3)$
7	$\Theta(1)$
Total	$\Theta(V ^3)$

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Claim:

Let $G = (V, E)$ be a weighted digraph with $w: E \rightarrow \mathbb{R}$.

After the **k-th** iteration of the for-loop in line 3 of Floyd-Warshall's algorithm, it holds that:

for every $i, j \in V$, $d_{i,j}^k$ is the weight of a shortest path from i to j that may use intermediate vertices from the set: $\{1, \dots, k\}$.

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Proof (of claim): By induction on k .

Base Case: $k=0$

In this case, k is less than the lowest valid vertex value.
Therefore, no intermediate vertices can be used before the first iteration.

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Proof (of claim): By induction on k .

Base Case: $k=0$

In this case, k is less than the lowest valid vertex value. Therefore, no intermediate vertices can be used before the first iteration.

Indeed, before the first iteration $d_{i,j}^0$ values are defined in line 2 to be the weight of direct edge from i to j (if exists), for every $i, j \in V$. Therefore, $d_{i,j}^0$ is indeed the weight of shortest direct path.

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Proof (cont.):

Induction step:

Assume by induction that after the $(k-1)$ -th iteration of line 3, for every $i, j \in V$, $d_{i,j}^{k-1}$ is the weight of a shortest path from i to j that may use intermediate vertices from the set: $\{1, \dots, k-1\}$.

We prove that after the k -th iteration of line 3, for every $i, j \in V$, $d_{i,j}^k$ is the weight of a shortest path from i to j that may use intermediate vertices from the set: $\{1, \dots, k\}$.

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Proof (cont.):

There are two possibilities for a shortest path from i to j that may use intermediate vertices from the set $\{1, \dots, k\}$:

1. either it does not use the vertex k , or
2. it uses the vertex k .

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Proof (cont.):

There are two possibilities for a shortest path from i to j that may use intermediate vertices from the set $\{1, \dots, k\}$:

1. either it does not use the vertex k , or
2. it uses the vertex k .

In case 1, the shortest path from i to j that may use intermediate vertices from the set $\{1, \dots, k\}$ is actually a shortest path from i to j that uses intermediate vertices only from the set $\{1, \dots, k - 1\}$.

Therefore, by induction hypothesis its value is $d_{i,j}^{k-1}$.

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Proof (cont.):

In case 2, shortest such path has 2 parts:

$i \rightsquigarrow k \rightsquigarrow j$

By sub-paths optimality
of shortest paths

Which intermediate vertices
used in sub-paths ?

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Proof (cont.):

In case 2, shortest such path has 2 parts:

$i \rightsquigarrow k \rightsquigarrow j$

By induction hypothesis, we have that

$d_{i,k}^{k-1}$ contains the weight of shortest path

from i to k that may use intermediate vertices from the set

$\{1, \dots, k-1\}$ after the $(k-1)$ -th iteration. Similarly, $d_{k,j}^{k-1}$ contains the weight of shortest path from k to j that may use intermediate vertices from the set $\{1, \dots, k-1\}$ after the $(k-1)$ -th iteration.

By sub-paths optimality
of shortest paths

Which intermediate vertices
used in sub-paths ?

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Proof (cont.):

By the definition of path-weight the total path weight is the sum.



APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Proof (cont.):

By the definition of path-weight the total path weight is the sum.



Since the computation in line 6 assigns to $d_{i,j}^k$ the better weight of the two cases 1 and 2, we have that after the k -th iteration of line 3, for every $i, j \in V$, $d_{i,j}^k$ is the weight of a shortest path from i to j that may use intermediate vertices from the set: $\{1, \dots, k\}$.

APSP- All Pairs Shortest Paths

Floyd-Warshall's Algorithm Correctness

Conclusion:

Let $G = (V, E)$ be a weighted digraph with $w: E \rightarrow \mathbb{R}$.

When Floyd-Warshall algorithm halts, we have that:

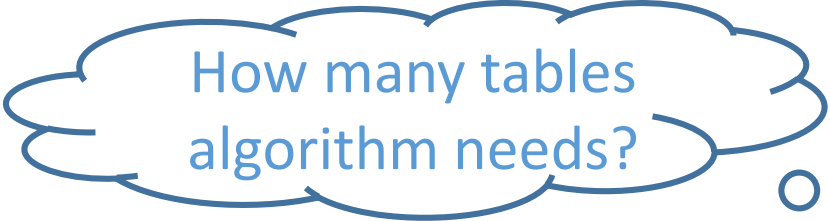
for every $i, j \in V$, $D^n[i, j] = \delta(i, j)$.

Proof (of conclusion):

Take $k=n$ in the last claim, and conclusion follows (why?).



What is the **space** complexity?



How many tables algorithm needs?